Network Switch Test Automation

802.1Q VLAN Bridging, LLDP and STP - explained in simple terms

The resume line this document explains:

> Raised test coverage from 55% to 90% and cut regression cycle time 60% by building Python and Robot Framework suites that automate unit and integration test cases for 802.1Q VLAN bridging, LLDP, and STP features.

What is inside:

1. Overall Summary (in simple terms)
2. Background - the networking ideas you need first
3. Components - every part of the system, end to end
4. Workflow - every step of a test run, end to end
5. Architecture Diagram - every box, line and number explained
6. How the numbers (55% to 90%, 60% faster) were achieved
7. Small code samples
8. Glossary

> **Note:** This document describes a typical, realistic design for a project like this one. Tool names such as Jenkins, Netmiko, Scapy and pabot are common choices; if your project used different tools (for example GitLab CI instead of Jenkins), simply swap the names - the ideas stay the same.

1. Overall Summary (in Simple Terms)

A network switch is the box that connects computers in an office or data center and passes data between them. Before a new version of switch software is released, a team must check that its features still work. Three important features are:

- **VLANs (802.1Q)** - splitting one physical switch into several separate 'virtual' networks.
- **LLDP** - switches introducing themselves to their neighbors ('Hi, I am Switch-2, port 5').
- **STP** - stopping data from running around in circles forever when cables form a loop.

Checking these by hand is slow: an engineer logs in to each switch, types commands, sends test traffic, reads the output and writes down pass or fail. Doing this for hundreds of test cases every release takes days, and many tests simply never got run.

This project replaced that manual work with a 'robot tester'. Test cases were written in Robot Framework (easy-to-read, English-like test files) and powered by Python code that actually logs in to the switches, configures them, sends real network frames and checks the results. A CI server (Jenkins) runs everything automatically every night or whenever code changes, and publishes a report.

> **In one sentence:** We taught a computer to do the switch testing that people used to do by hand, so more tests get run (coverage up from 55% to 90%) and the full set finishes much faster (regression time down 60%).

The two results, in plain words

Result	Before	After	What it means
Test coverage	55%	90%	Out of every 100 test cases written in the test plan, only 55 were being checked automatically. Now 90 are.
Regression cycle time	100% (baseline)	40% of baseline	Re-checking everything after a change now takes 60% less time. Example: 5 working days became 2.

Why it matters

- **Bugs are caught earlier** - a broken VLAN or STP change is found the next morning, not weeks later.
- **Releases go faster** - the team is no longer waiting days for manual re-testing.
- **Results are trustworthy** - a machine runs the same steps the same way every time, with full logs.
- **Engineers do more valuable work** - they design new tests instead of repeating old ones.

2. Background - The Ideas You Need First

2.1 802.1Q VLAN bridging

Simple idea: one switch, many separate networks. Imagine an apartment building (the switch) with many flats (VLANs). People in flat 10 cannot walk into flat 20 even though they share the same building.

- **VLAN ID** - a number from 1 to 4094 that names each virtual network (0 and 4095 are reserved).
- **Access port** - a port that belongs to just one VLAN. A normal PC plugs in here and does not know about VLANs.
- **Trunk port** - a port that carries many VLANs between switches. Each frame gets a small 4-byte label called the **802.1Q tag** that says which VLAN it belongs to.
- **The tag** contains: TPID 0x8100 (says 'this frame is tagged'), a 3-bit priority (PCP), a 1-bit DEI flag, and the 12-bit VLAN ID.
- **Native VLAN / PVID** - the VLAN used for frames that arrive with no tag.
- **Bridging** - the switch learns which MAC address lives on which port (per VLAN) and forwards frames only where needed.

2.2 LLDP (Link Layer Discovery Protocol)

Simple idea: every few seconds each switch shouts to its direct neighbor, 'I am Switch-1, this is my port 3, my IP is 10.0.0.1.' The neighbor writes this down in a table.

- **TLV** (Type-Length-Value) - each piece of information is packed as a TLV. Mandatory: Chassis ID, Port ID, Time-To-Live. Optional: system name, description, capabilities, management address.
- **Timers** - transmit interval (default 30 s) and hold multiplier (default 4). TTL = interval x multiplier = 120 s. If no message arrives within TTL, the neighbor entry is removed.
- **Modes** - a port can send only, receive only, both, or neither.

2.3 STP (Spanning Tree Protocol)

Simple idea: if switches are cabled in a circle, data can loop forever and crash the network (a 'broadcast storm'). STP finds the loop and switches off one link, keeping it as a spare. If the main link breaks, the spare is switched back on.

- **Root bridge** - the 'boss' switch, chosen by the lowest Bridge ID (priority first, then MAC address).
- **Port roles** - Root port (best path to the boss), Designated port (forwards for a segment), Alternate/Backup port (the blocked spare).
- **Port states** - classic STP: Blocking, Listening, Learning, Forwarding. Rapid STP (RSTP): Discarding, Learning, Forwarding.
- **BPDU** - the small 'hello' messages switches exchange to run STP.
- **Convergence** - the time needed to recover after a link fails (around 30-50 s for STP, often under a few seconds for RSTP).
- **Protection features** - BPDU guard, root guard, edge (PortFast) ports.

2.4 Testing words

Term	Simple meaning
Unit test	A small test that checks one thing on its own. Here: one feature on one switch (e.g. 'can I create VLAN 20?') and also checks of the Python helper code itself.
Integration test	A bigger test that checks several parts working together: several switches, or several features at once (e.g. 'does STP still block the loop while VLAN 20 traffic flows over the trunk?').
Test plan	The master list of every test case the team agreed should be checked.
Test coverage	Automated test cases divided by total test cases in the plan, as a percentage.
Regression	Re-running old tests after a change to make sure nothing that used to work is now broken.
Regression cycle time	How long one full regression takes from start to final report.
DUT	Device Under Test - the switch being tested.

3. Components - Every Part, End to End

The system has four layers. Each component below has an ID (A1, B4, C2...) that matches the box with the same ID in the architecture diagram in Section 5.

Layer	Components	Job of the layer
1. Trigger	A1 Engineer, A2 Git Repository, A3 CI Server	Decide when a test run starts
2. Test Framework	B1 Robot Runner, B2 Pytest, B3 Topology, B4 Suites, B5 Resources, B6 Python Libraries	Decide what to test and how
3. Testbed	C1 Management Network, C2 Switches (DUTs), C3 Traffic Hosts	The real equipment being tested
4. Results	D1 Robot Outputs, D2 Coverage Tracker, D3 Dashboard + Alerts	Tell people what happened

Layer 1 - Trigger

A1. Engineer and Test Plan

What it is: the people who write test cases and the test plan document that lists everything that should be tested.

> **Think of it like:** a teacher writing the exam questions before the exam.

What it does, step by step:

- Reads the feature specification (for example the IEEE 802.1Q, 802.1AB (LLDP) and 802.1D/802.1w (STP) standards and the product requirements).
- Writes test cases in the test plan, each with an ID such as TC-VLAN-012, steps, and expected result.
- Turns each test case into automation: a Robot test case that carries the same ID as a tag.
- Pushes the code to Git and reviews colleagues' code through pull requests.

A2. Git Repository

What it is: the shared online folder that stores all test code with full history of every change.

> **Think of it like:** a shared notebook where every edit is saved with the writer's name and date.

What it does, step by step:

- Holds test suites, keywords, Python libraries, topology files, unit tests and the Jenkinsfile in one place.
- Branches let people work without disturbing each other; pull requests make sure someone reviews changes.
- Every push sends a signal (a 'webhook') to Jenkins so a quick test run can start automatically.
- Folder layout (shown below) keeps each feature in its own place.

Example repository layout

```
network-tests/
|-- suites/
|   |-- vlan/      vlan_access.robot, vlan_trunk.robot, vlan_negative.robot
|   |-- lldp/      lldp_neighbors.robot, lldp_tlv.robot, lldp_timers.robot
|   |-- stp/       stp_root.robot, stp_roles.robot, stp_failover.robot
|   `-- integration/  vlan_stp_lldp_combined.robot
|-- resources/     common.resource, vlan.resource, lldp.resource, stp.resource
|-- libraries/     DeviceLib.py, ConfigLib.py, ParserLib.py, TrafficLib.py
|-- topologies/    lab1.yaml, lab2.yaml
|-- unit_tests/    test_parser.py, test_config.py
|-- tools/         coverage_report.py
|-- Jenkinsfile
`-- requirements.txt
```

A3. CI Server (Jenkins)

What it is: an always-on computer that automatically starts the tests, at night or after every code change.

> **Think of it like:** an alarm clock plus a manager: it wakes up at the right time and tells everyone to start work.

What it does, step by step:

- **Triggers:** a nightly timer runs the full regression; a code push runs a short 'smoke' set (most important tests only).
- **Jenkinsfile:** a script stored in Git that lists the stages - Checkout, Unit Tests, Reserve Testbed, Run Robot, Publish, Release.
- **Parameters:** lets a person choose which feature (vlan / lldp / stp / all) and which lab to use when starting a run by hand.
- **Agent:** Jenkins sends the job to the control PC that is wired to the lab.
- **After the run:** stores the reports as 'artifacts', draws pass/fail trend charts and sends emails or chat messages.

Layer 2 - Test Framework (runs on the control PC)

B1. Robot Framework Runner (robot / pabot)

What it is: the program that reads the .robot test files and executes them one step at a time.

> **Think of it like:** a stage director reading a script and telling each actor when to speak.

What it does, step by step:

- Started by Jenkins with a command such as: robot --include vlan --variablefile topologies/lab1.yaml suites/
- **Tags** choose what to run: smoke, vlan, lldp, stp, integration, or a test-plan ID.
- **pabot** (parallel Robot) runs several suites at the same time on different labs - a big reason the run became faster.
- **--rerunfailed** re-runs only the failed tests to separate real bugs from one-off glitches.
- **Listeners** can push live progress to Jenkins while the run is happening.
- Writes three files at the end: output.xml, log.html and report.html (see D1).

B2. Pytest Unit Tests

What it is: small, very fast Python tests that check the helper code itself, without any real switch.

> **Think of it like:** testing the thermometer before you use it to check a patient.

What it does, step by step:

- Feeds saved sample outputs (for example the text of 'show vlan brief') into ParserLib and checks the result is correct.
- Uses 'mocks' (fake switches) to check that DeviceLib and ConfigLib send the right commands in the right order.
- Checks protocol logic helpers, e.g. a function that predicts which switch should become STP root from priorities and MACs.
- Runs in seconds as the first Jenkins stage. If it fails, the expensive hardware run is skipped - saving lab time.
- Has its own code coverage report (pytest-cov) to show how much of the Python code is exercised.

B3. Topology and Variables Files (YAML)

What it is: a plain-text 'map' of the lab: which switches exist, how to log in, and which port is cabled to what.

> **Think of it like:** the seating chart a teacher uses so tests work in any classroom.

What it does, step by step:

- Lists every switch with its management IP, login method (SSH or console) and device type.
- Lists the cables: DUT1 port 1 to DUT2 port 1, DUT2 port 2 to DUT3 port 2, and so on.
- Lists test data: VLAN IDs, STP priorities, LLDP timer values.
- Because tests read names like ${DUT1_TRUNK_PORT} instead of fixed values, the same tests run on any lab by changing only this file.
- Passwords are not kept here; they come from Jenkins' secure credential store at run time.

Example topologies/lab1.yaml

```
devices:
  DUT1: { host: 10.10.0.11, type: switch_os, protocol: ssh }
  DUT2: { host: 10.10.0.12, type: switch_os, protocol: ssh }
  DUT3: { host: 10.10.0.13, type: switch_os, protocol: ssh }
links:
  - [DUT1:eth1, DUT2:eth1]      # trunk
  - [DUT2:eth2, DUT3:eth2]      # trunk
  - [DUT1:eth3, DUT3:eth3]      # redundant link (forms STP loop)
hosts:
  HOST_A: { mgmt: 10.10.0.21, test_nic: ens4, connected_to: DUT1:eth10 }
  HOST_B: { mgmt: 10.10.0.22, test_nic: ens4, connected_to: DUT3:eth10 }
test_data:
  vlans: [10, 20, 30]
  stp_root_priority: 4096
```

B4. Test Suites (.robot files)

What it is: the actual test cases, written in near-plain English, grouped into one folder per feature.

> **Think of it like:** a recipe book - each recipe (test case) lists simple steps anyone can follow.

What it does, step by step:

- Each file has four parts: **Settings** (which libraries/resources to load, setup and teardown), **Variables**, **Test Cases**, and local **Keywords**.
- **Suite Setup / Teardown** run once per file (log in, save config / restore config, log out).
- **Test Setup / Teardown** run around every test so each test starts clean.
- **Test Templates** make data-driven tests: write one test, feed it a table of 50 VLAN IDs - this multiplied coverage cheaply.
- **VLAN 802.1Q suite covers:**
  - create / delete VLANs; valid IDs 1-4094; reject invalid IDs 0, 4095, 5000
  - access port sends untagged frames; trunk port adds the correct 802.1Q tag
  - native VLAN / PVID handling; allowed-VLAN list on trunks
  - isolation - traffic in VLAN 10 never reaches VLAN 20; MAC learning per VLAN
  - scale (maximum VLANs) and configuration surviving a reboot
- **LLDP suite covers:**
  - enable / disable globally and per port; neighbor appears in the table
  - mandatory TLVs (Chassis ID, Port ID, TTL) and optional TLVs (system name, capabilities, management address)
  - timers - interval x hold multiplier = TTL; neighbor ages out after the link goes down
  - transmit-only and receive-only modes
- **STP / RSTP suite covers:**
  - root bridge election (lowest priority, then lowest MAC); re-election when priority changes
  - port roles and states; the redundant link is blocked so no loop forms
  - failover - cut the active link, measure convergence time, confirm traffic recovers
  - BPDU guard, root guard and edge (PortFast) ports
- **Integration suite covers:** features together, e.g. STP per VLAN over trunks, LLDP neighbors correct while STP blocks a port, VLAN traffic continuing after STP failover.

B5. Resource Files (shared keywords)

What it is: reusable 'action words' such as Create VLAN, Verify LLDP Neighbor or Verify STP Root, written once and used by every test.

> **Think of it like:** LEGO bricks - build many different tests from the same small set of pieces.

What it does, step by step:

- Each keyword hides the details. A test just says: Create VLAN ${DUT1} 20.
- Inside, the keyword calls the Python libraries to send commands and check the answer.
- Includes waiting keywords for protocols that need time, e.g. Wait Until Keyword Succeeds 60s 5s STP Port Should Be Forwarding.
- Fixing a keyword once fixes every test that uses it - this is what kept maintenance cheap as coverage grew.

B6. Python Keyword Libraries

What it is: the Python code that does the real work: talking to switches, sending traffic and reading answers.

> **Think of it like:** the hands and eyes of the robot tester.

What it does, step by step:

- **DeviceLib** - opens SSH (e.g. with Netmiko) or console sessions, sends commands, waits for the prompt, handles timeouts, reconnects after a reboot, and logs every command and reply.
- **ConfigLib** - pushes configuration blocks, saves a 'snapshot' before each test and rolls back afterwards so tests never disturb each other.
- **ParserLib** - turns messy CLI text into clean data (using regular expressions or TextFSM templates), e.g. 'show spanning-tree' becomes {root: DUT1, eth3: alternate/blocking}.
- **TrafficLib** - uses Scapy on the traffic hosts to build Ethernet frames (with or without an 802.1Q tag), send them, capture what arrives, and count or inspect them. Also saves .pcap files for debugging.
- Every public Python function automatically becomes a Robot keyword (e.g. def send_tagged_frame() becomes Send Tagged Frame).

Layer 3 - Testbed

C1. Management Network

What it is: a separate 'back door' network used only to log in to and control the switches.

> **Think of it like:** a staff-only corridor that stays open even when the main hallway is blocked.

What it does, step by step:

- Every switch and host has a management port on this network.
- Needed because tests deliberately break the data network (STP blocks ports, links are cut). The control PC must still reach the switches.
- A console server gives a last-resort connection if a switch loses its IP (for example after a bad config or a reboot).

C2. Switches Under Test (DUT-1, DUT-2, DUT-3)

What it is: the three real (or virtual) switches running the software being tested.

> **Think of it like:** the patients in a check-up - everything else exists to examine them.

What it does, step by step:

- **Why three switches?** Three switches cabled in a triangle form a loop, which is needed to test STP. The middle switch also lets VLAN traffic 'travel through' a switch (transit).
- DUT1-DUT2 and DUT2-DUT3 are trunk links carrying several VLANs.
- DUT1-DUT3 is the redundant link. STP must block one port so the triangle does not loop.
- Can be physical hardware or virtual switch images in a lab server - same tests work on both.

C3. Traffic Hosts (Host A and Host B)

What it is: two Linux machines connected to access ports, used to send and catch test frames.

> **Think of it like:** one person throws a ball (Host A) and another checks whether it was caught and what it looks like (Host B).

What it does, step by step:

- Host A sends frames in a chosen VLAN; Host B listens and records what arrives.
- Proves real forwarding, not just that the configuration 'looks' right.
- Example checks: a VLAN 10 frame reaches Host B only if Host B is in VLAN 10; frames on the trunk carry tag 10; no frames are duplicated (no loop).
- During STP failover tests, Host A sends a steady stream; lost frames show how long recovery took.

Layer 4 - Results and Feedback

D1. Robot Outputs

What it is: the files Robot Framework writes after every run.

> **Think of it like:** a detailed school report card plus the teacher's marked answer sheet.

What it does, step by step:

- **output.xml** - the machine-readable record of every step; used by other tools.
- **log.html** - step-by-step detail with timestamps, every command sent and every reply received. First place to look when a test fails.
- **report.html** - one-page summary: how many passed, failed, by tag and by suite.
- **xunit.xml** - a format Jenkins understands for its trend charts.
- Extra attachments on failure: .pcap captures and the switch's own logs ('show logging').

D2. Coverage Tracker

What it is: a small script that compares the test plan with the tests that were actually automated and run.

> **Think of it like:** a checklist where every finished item is ticked off.

What it does, step by step:

- Reads output.xml and collects the test-plan ID tags (TC-VLAN-012 and so on).
- Compares them with the full list of IDs in the test plan.
- Coverage = automated test cases / total test cases x 100.
- Example: plan of 400 cases. Before: 220 automated = 55%. After: 360 automated = 90%.
- Lists which cases are still manual, so the team knows what to automate next.

D3. Dashboard and Alerts

What it is: the place where people see results without opening any files.

> **Think of it like:** a scoreboard at a stadium plus a phone notification when your team scores.

What it does, step by step:

- Jenkins trend charts: pass rate per night, duration per run, coverage over time.
- Email / chat alerts on failure with a direct link to log.html.
- Green build = everyone can move on; red build = an engineer investigates, files a bug or fixes the test.

4. Workflow - Every Step, End to End

This is what happens in one full test run, from the moment it starts until people see the result. The flowchart below shows the whole run; each step is then explained in detail.

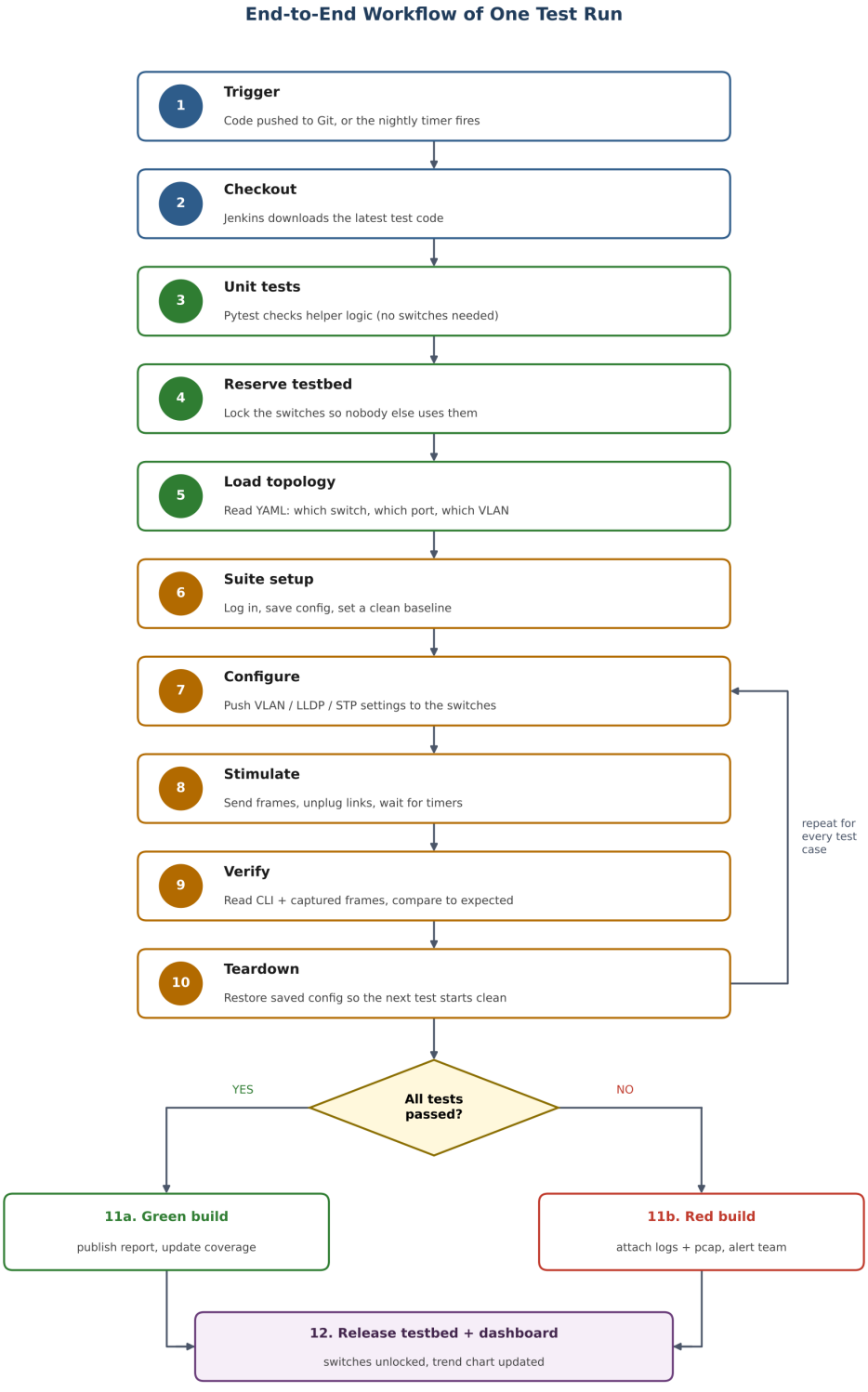


Figure 1 - End-to-end workflow of one test run

Step 1 - Trigger

A run starts in one of three ways.

- **Nightly timer** - Jenkins starts the full regression at a fixed time (e.g. 1:00 AM).
- **Code push** - a change in Git sends a webhook and Jenkins starts the short smoke set.
- **Manual** - an engineer clicks 'Build with Parameters' and picks feature + lab.

Step 2 - Checkout

Jenkins gets the latest code onto the control PC.

- Clones (downloads) the Git repository at the exact commit that triggered the run.
- Creates a fresh Python virtual environment and installs requirements.txt (robotframework, pabot, netmiko, scapy, pyyaml, pytest).
- Records the commit ID so every result can be traced back to the exact code.

Step 3 - Unit tests

Fast safety check of the test code itself.

- Runs pytest on unit_tests/ - usually finishes in under a minute.
- If any unit test fails, the pipeline stops here, so broken test code never wastes hours of lab time.

Step 4 - Reserve testbed

Make sure nobody else is using the lab.

- Jenkins takes a 'lock' on the lab (e.g. with the Lockable Resources plugin).
- Pings every switch and host on the management network to confirm the lab is healthy.
- If a device is unreachable, the run is marked as a lab problem, not a product bug.

Step 5 - Load topology

Tell the tests which lab they are on.

- Robot reads the YAML file: device addresses, cable map and test data.
- Values become variables like ${DUT1}, ${DUT1_TRUNK_PORT}, ${VLAN_LIST}.

Step 6 - Suite setup

Prepare the switches once per test file.

- DeviceLib logs in to every switch over SSH.
- Records software version and saves the current configuration as a baseline snapshot.
- Applies the common base configuration (e.g. hostnames, trunk ports up, LLDP on).

Step 7 - Configure

Set up exactly what this test case needs.

- Example (VLAN test): create VLAN 20, make DUT1 eth10 an access port in VLAN 20, allow VLAN 20 on the trunks.
- Example (STP test): set DUT1 bridge priority to 4096 so it should become root.
- ConfigLib checks each command was accepted (no error message from the switch).

Step 8 - Stimulate

Make something happen on the network.

- Send frames from Host A with TrafficLib (tagged or untagged, broadcast or unicast).

- Shut a link down to force STP failover or LLDP ageing.
- Wait for protocol timers using 'wait until' keywords instead of fixed sleeps - waits only as long as needed, which saves time.

Step 9 - Verify

Compare what happened with what should happen.

- Read switch state: show vlan, show lldp neighbors, show spanning-tree - ParserLib turns it into data.
- Read Host B's capture: did the frame arrive? With which tag? How many copies?
- Assertions such as 'Should Be Equal ${root} DUT1'. Any mismatch marks the test FAIL and saves evidence (logs, pcap).

Step 10 - Teardown

Clean up so the next test starts fresh.

- ConfigLib rolls back to the saved snapshot; links shut in Step 8 are brought back up.
- Runs even if the test failed, so one failure never breaks the following tests.
- Then the loop returns to Step 7 for the next test case until the suite is finished.

Step 11 - Decide: all passed?

Robot finishes and writes output.xml, log.html and report.html.

- Optionally re-runs only failed tests once (--rerunfailed) and merges results (rebot), to filter out lab glitches.
- **11a Green build** - publish reports, update coverage numbers.
- **11b Red build** - attach logs and pcaps, alert the team by email/chat; an engineer checks if it is a product bug (file a ticket) or a test problem (fix the test).

Step 12 - Release and report

Close out the run.

- Unlock the lab so another run can use it.
- Coverage script updates the coverage figure; dashboard trend charts update.
- Results are kept in Jenkins history so the team can compare nights and spot when something started failing.

Worked example: one VLAN test traced end to end

Test case TC-VLAN-012: 'A frame sent in VLAN 20 from Host A crosses two trunks with tag 20 and arrives untagged at Host B.'

1. **Configure:** create VLAN 20 on DUT1, DUT2 and DUT3; set DUT1 eth10 and DUT3 eth10 as access ports in VLAN 20; allow VLAN 20 on both trunks.
2. **Stimulate:** Host A sends 100 broadcast frames with no tag.
3. **On the wire:** DUT1 adds an 802.1Q tag (0x8100, VLAN 20) when sending on the trunk; DUT2 forwards it; DUT3 removes the tag on its access port.
4. **Verify:** Host B captured exactly 100 frames, all untagged; 'show mac address-table vlan 20' on DUT3 lists Host A's MAC; a host in VLAN 30 received zero frames (isolation).
5. **Teardown:** remove VLAN 20 and restore the snapshot. Result: PASS, logged under tag TC-VLAN-012, which counts toward coverage.

Worked example: one STP failover test

1. **Configure:** set DUT1 priority 4096 (root). Expect DUT3 eth3 (redundant link) to be Alternate / Blocking.
2. **Verify before:** parser confirms root = DUT1 and the expected blocked port. Host B sees no duplicate frames (no loop).
3. **Stimulate:** Host A sends a continuous stream; the test shuts the DUT1-DUT2 trunk.
4. **Verify after:** within the allowed time (e.g. under 3 s for RSTP) the blocked port becomes Forwarding and traffic resumes. Lost frames divided by send rate gives the measured convergence time.
5. **Teardown:** bring the trunk back, confirm the original port roles return, restore the snapshot.

5. Architecture Diagram - Every Detail Explained

The full diagram is below. It is drawn as four stacked layers, read from top to bottom: something starts the run (Layer 1), the framework decides what to do (Layer 2), the switches are tested (Layer 3), and results are reported (Layer 4).

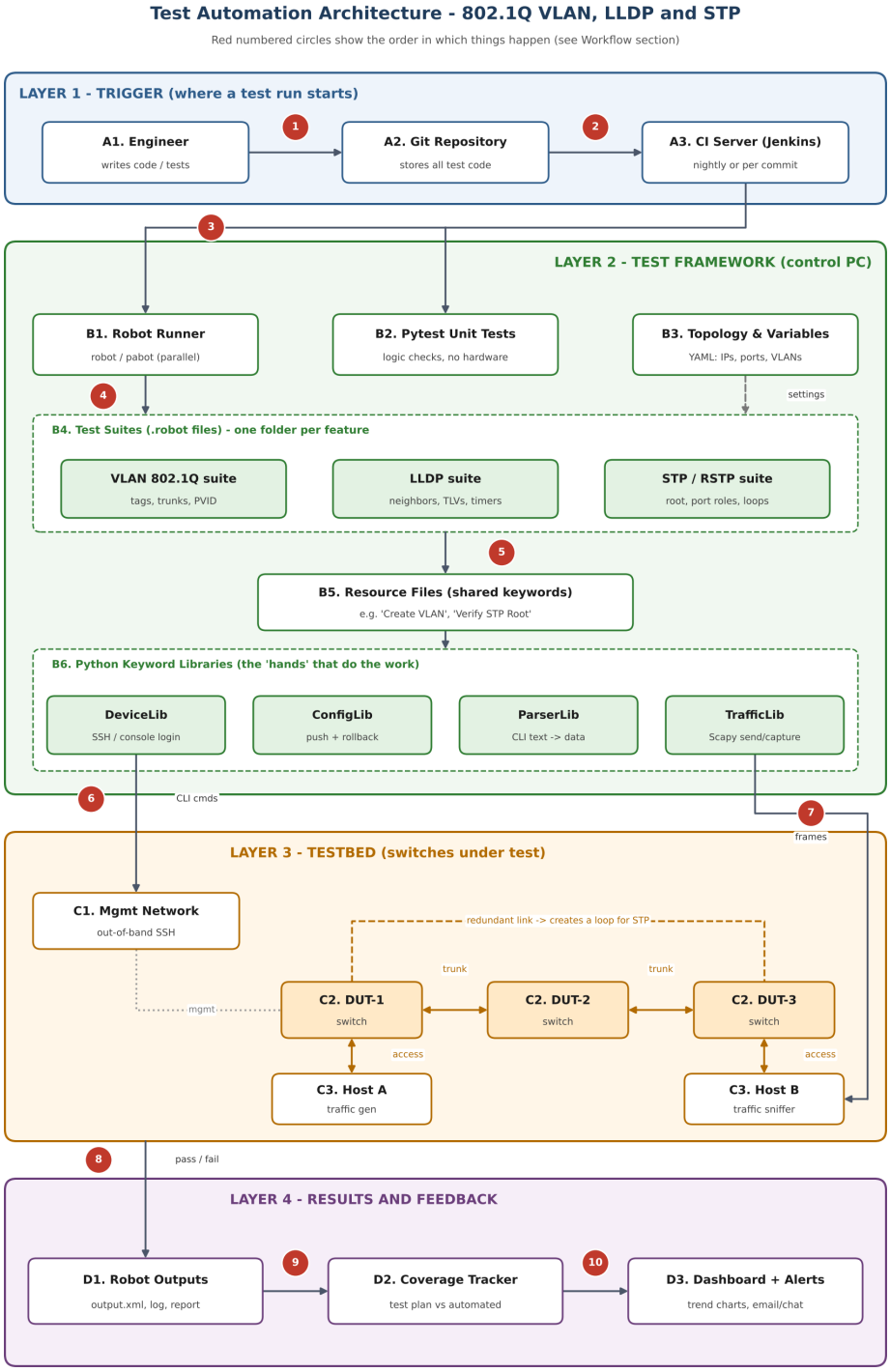


Figure 2 - Test automation architecture

5.1 How to read the diagram

Visual element	What it means
Large coloured band	A layer. Blue = Trigger, green = Test framework, orange = Testbed, purple = Results.
White rounded box	One component. Its ID (A1, B4, C2...) matches Section 3.
Light-filled box inside a dashed frame	A member of a group: the three feature suites (B4) or the four Python libraries (B6).
Solid grey arrow	Control or data flows this way (e.g. Jenkins starts Robot).
Dashed grey arrow	Supporting input, not a main step (B3 settings feed the suites).
Orange double-headed arrow	A real network cable; frames travel both ways.
Orange dashed line	The redundant link DUT1-DUT3 that creates the loop STP must block.
Grey dotted line	Management connection (control only, no test traffic).
Red numbered circle	The order of events. Numbers follow the run from start to finish.

5.2 The numbered arrows, in order

No.	From -> To	What travels	Simple meaning
1	A1 Engineer -> A2 Git	test code (git push)	The engineer saves new tests to the shared folder.
2	A2 Git -> A3 Jenkins	webhook / nightly timer	Jenkins is told 'time to test'.
3	A3 Jenkins -> B1 Runner and B2 Pytest	start command + parameters	Jenkins runs unit tests and then starts Robot.
4	B1 Runner -> B4 Suites	selected .robot files (by tag)	Robot opens the chosen test files.
5	B4 Suites -> B5 Resources	keyword calls	Each test step calls a shared action word.
-	B5 Resources -> B6 Libraries	Python function calls	Each action word runs Python code.
6	B6 DeviceLib -> C1 Mgmt -> C2 DUTs	CLI commands over SSH	Python logs in and configures / reads the switches.
7	B6 TrafficLib -> C3 Hosts	frames to send, capture requests	Python tells the hosts to send and catch frames.
8	Layer 3 -> D1 Outputs	pass / fail + evidence	What happened in the lab is written to the report files.
9	D1 Outputs -> D2 Coverage	output.xml with test IDs	Coverage is calculated.
10	D2 Coverage -> D3 Dashboard	numbers + trends	People see results and get alerts.

5.3 Box-by-box walk-through

Layer 1 - Trigger (blue)

- **A1 Engineer** - writes tests and pushes them (arrow 1).
- **A2 Git Repository** - stores everything and notifies Jenkins (arrow 2).
- **A3 CI Server (Jenkins)** - the line leaving its bottom splits in two: one branch to B1 Robot Runner, one to B2 Pytest (arrow 3).

Layer 2 - Test Framework (green)

- **B1 Robot Runner** - top-left; its arrow (4) goes down into the suites frame.
- **B2 Pytest Unit Tests** - top-middle; runs first and stands alone because it needs no hardware.
- **B3 Topology & Variables** - top-right; dashed 'settings' arrow into the suites shows it only supplies values.
- **B4 Test Suites** - dashed frame holding VLAN 802.1Q, LLDP and STP/RSTP suites side by side, showing they are parallel and independent.
- **B5 Resource Files** - one centre box, because all three suites share the same keywords (arrow 5).
- **B6 Python Libraries** - dashed frame with DeviceLib, ConfigLib, ParserLib and TrafficLib. Only DeviceLib and TrafficLib have arrows leaving the layer, because they are the two that touch the lab; ConfigLib and ParserLib work through DeviceLib.

Layer 3 - Testbed (orange)

- **C1 Management Network** - arrow 6 from DeviceLib lands here; a dotted 'mgmt' line connects it to the switches.
- **C2 DUT-1, DUT-2, DUT-3** - in a row, joined by two 'trunk' cables. The dashed line above them is the redundant link that closes the loop.
- **C3 Host A** - under DUT-1 on an 'access' cable; sends traffic.
- **C3 Host B** - under DUT-3 on an 'access' cable; captures traffic. Arrow 7 from TrafficLib reaches it along the right edge (it also controls Host A).

Layer 4 - Results (purple)

- **D1 Robot Outputs** - receives arrow 8 (results from the lab run).
- **D2 Coverage Tracker** - fed by arrow 9.
- **D3 Dashboard + Alerts** - fed by arrow 10; this is where people see the outcome.

5.4 Physical links in the lab

Link	Type	Carries	Why it exists
DUT1 - DUT2	Trunk	VLANs 10, 20, 30 (tagged)	Tests tagging and transit forwarding
DUT2 - DUT3	Trunk	VLANs 10, 20, 30 (tagged)	Second hop; LLDP neighbor checks
DUT1 - DUT3	Redundant	Same VLANs	Forms the loop for STP blocking and failover tests
Host A - DUT1	Access	One VLAN (untagged)	Traffic source
Host B - DUT3	Access	One VLAN (untagged)	Traffic sink / capture point
Mgmt - all devices	Management	SSH only	Control path that survives test-induced outages

6. How the Numbers Were Achieved

6.1 Coverage: 55% to 90%

- **Reusable keywords (B5)** - once 'Create VLAN' or 'Verify STP Root' existed, each new test took minutes instead of hours.
- **Data-driven templates** - one template plus a data table produced dozens of cases (every valid and invalid VLAN ID, every STP priority combination).
- **Traffic verification (TrafficLib)** - made tests possible that were too hard by hand, such as counting frames or checking tag bytes.
- **Integration suites** - covered combined-feature cases that were usually skipped manually.
- **Test-plan ID tags + coverage script (D2)** - showed exactly which cases were still manual, so effort went to the gaps.

6.2 Regression time: 60% shorter

Change	Why it saves time
Machine runs overnight	No waiting for people; results are ready in the morning.
Parallel execution with pabot	VLAN, LLDP and STP suites run at the same time on separate labs.
Smart waits instead of fixed sleeps	A test continues as soon as STP converges, not after a fixed 60 s.
Automatic setup / teardown with snapshots	No manual re-cabling or re-configuring between tests.
Unit tests run first	Broken test code is caught in a minute, not after hours of lab time.
Re-run only failures	Glitches are retried in minutes instead of repeating the whole set.

> **Example maths:** If a full manual regression took 5 working days, a 60% reduction means it now takes 2 days (5 x 0.4 = 2). Use the real figures from your project when you talk about it.

7. Small Code Samples

These short examples show what each layer looks like in practice.

B4 - Robot test case (suites/vlan/vlan_trunk.robot)

```
*** Settings ***
Resource          ../../resources/vlan.resource
Suite Setup       Connect And Save Baseline    ${DUT1}    ${DUT2}    ${DUT3}
Suite Teardown    Restore Baseline And Disconnect
Test Teardown     Rollback To Snapshot

*** Test Cases ***
Tagged Frame Crosses Trunk In VLAN 20
    [Tags]    vlan    integration    TC-VLAN-012
    Create VLAN On All Switches          20
    Set Access Port    ${DUT1}    ${HOST_A_PORT}    20
    Set Access Port    ${DUT3}    ${HOST_B_PORT}    20
    Allow VLAN On Trunks                 20
    Start Capture      ${HOST_B}
    Send Frames        ${HOST_A}    count=100
    ${frames}=    Stop Capture    ${HOST_B}
    Should Be Equal As Integers    ${frames.count}    100
    Frames Should Be Untagged      ${frames}
```

B5 - Shared keyword (resources/vlan.resource)

```
*** Keywords ***
Create VLAN On All Switches
    [Arguments]    ${vlan_id}
    FOR    ${dut}    IN    ${DUT1}    ${DUT2}    ${DUT3}
        Apply Config    ${dut}    vlan ${vlan_id}
        ${vlans}=    Get VLAN Table    ${dut}
        Should Contain    ${vlans}    ${vlan_id}
    END
```

B6 - Python keyword library (libraries/TrafficLib.py)

```
from scapy.all import Ether, Dot1Q, sendp

class TrafficLib:
    def send_tagged_frame(self, iface, vlan_id, count=1):
        """Robot keyword: Send Tagged Frame"""
        frame = Ether(dst="ff:ff:ff:ff:ff:ff") / Dot1Q(vlan=int(vlan_id)) / b"TEST"
        sendp(frame, iface=iface, count=int(count), verbose=False)
```

B2 - Pytest unit test (unit_tests/test_parser.py)

```
def test_parse_stp_root():
    raw = open("samples/show_spanning_tree.txt").read()
    result = ParserLib().parse_stp(raw)
    assert result["root_bridge"] == "4096.0011.2233.4455"
    assert result["ports"]["eth3"]["role"] == "alternate"
```

8. Glossary

Term	Plain meaning
802.1Q	The standard for VLAN tagging on Ethernet.
802.1AB	The standard that defines LLDP.
802.1D / 802.1w	Standards for classic STP and Rapid STP (RSTP).
BPDU	Bridge Protocol Data Unit - STP's hello message.
CI	Continuous Integration - automatically building and testing on every change.
Convergence	The time a network needs to settle after a change.
DUT	Device Under Test.
Jenkins	A popular CI server.
Keyword	A named, reusable action in Robot Framework.
Netmiko	A Python library for SSH access to network devices.
pabot	A tool that runs Robot Framework tests in parallel.
pcap	A saved packet capture file, viewable in Wireshark.
PVID	Port VLAN ID - the VLAN given to untagged frames on a port.
Robot Framework	An open-source test framework that uses readable, keyword-based test files.
Scapy	A Python library for building, sending and capturing network packets.
Smoke test	A small set of the most important tests, run quickly.
TLV	Type-Length-Value - the way LLDP packs information.
TPID	Tag Protocol Identifier (0x8100) that marks a frame as VLAN-tagged.
Trunk	A link that carries many VLANs using tags.
YAML	A simple text format for settings and data.